This is a simple example that reads the value of `switchPin` assigning that value to the variable `tiltSwitch` and then tests the condition of `tiltSwitch`. If `tiltSwitch` is equal to the constant `HIGH`, then it will call the `digitalWrite()` function to change the state of the digital pin 13 to `HIGH`, which would have the effect of turning on the pin 13 LED on the Arduino interface board. Notice that this is another example of a simple statement that does not require curly braces.

If we wanted the LED to only turn on when the switch is activated and to remain off the rest of the time, we would use an `if…else` statement. Take the following example:

```
tiltSwitch = digitalRead(switchPin);
if (tiltSwitch == HIGH) digitalWrite(13, HIGH);
else digitalWrite(13, LOW);
```

Like the previous example, this statement will turn on the LED if the switch has been activated, however, by using the else keyword it is possible to turn off the LED the rest of the time. Essentially, if the condition is true, only the first statement will be executed and if it is false, only the second statement will be executed. In plain English, this statement would be similar to saying

```
"If the switch is on then turn on the LED, otherwise turn off the LED."
```

So far, our `if` statements have been checking for a specific condition using the `==` (is equal to) operator although any of the comparative or logical operators as shown could be used instead. Take for example, the following:

```
if (myValue >= 10 && myValue <= 20) {
  statements
}
```

This `if` statement will check for a range of values, only executing the following statements if the value of `myValue` is somewhere between 10 and 20.

```
if (!myValue) {
  statements
}
```

This sample uses the `!` (not) operator so that the enclosed statements will only be executed if the variable `myValue` is false, which also corresponds to `0` or `LOW`, but we will get to that a little later in the next chapter.

For

The `for` statement, or `for` loop, is an iterative statement that allows the Arduino to repeatedly execute lines of code in a loop a specified number of times. What makes the `for` loop unique is that it is based on a counter, or a specific variable that is incremented each time the loop is repeated. This counter is even quite useful, as the counter itself can be used just like any other variable by statements that reside inside the loop, as you will see in later examples. The basic syntax of a `for` loop follows:

```
for (declaration; condition; increment) {
  statements
}
```

The `for` loop begins with three statements that include: variable declaration, conditional statement, and incremental statement. Of these, the first is the counter variable declaration or initialization and is run only once the first time through the loop. The second statement is the conditional statement using comparative operators just like those found in the `if` statement and is tested each time through the loop.